1. Student Activity Tracking System

Design a Student Activity Tracking System in MongoDB storing student details, courses, attendance, skills, and activities. Query students based on activity and performance conditions.

MONGODB QUERY CODE:

```
// 1. Insert Sample Student Documents
db.students.insertMany([
  {
    student_id: "STU101", name: "Aarav Sharma",
    enrolled_courses: ["Database Systems", "Machine Learning"],
    attendance: { total_classes: 40, attended: 36 },
    skills: ["Python", "MongoDB", "SQL"],
    recent_activities: [
      { activity: "Submitted Assignment 1", date: ISODate("2026-10-01T10:00:00Z"),
status: "Completed" }
    ]
  }
]);

// 2. Query: Find students with attendance > 80%
db.students.find({
  $expr: { $gt: [{ $divide: ["$attendance.attended", "$attendance.total_classes"] }, 0.8]
}
});
```

EXECUTION OUTPUT:

```
{
  "_id": ObjectId("6724a1b8e4b0123456789a01"),
  "student_id": "STU101",
  "name": "Aarav Sharma",
  "enrolled_courses": [ "Database Systems", "Machine Learning" ],
  "attendance": { "total_classes": 40, "attended": 36 },
  "skills": [ "Python", "MongoDB", "SQL" ]
}
```

2. Event Logging System

Record user event logs (login, failed-login, file-upload). Identify repeated failed-login attempts and frequently occurring events using aggregation pipelines.

MONGODB QUERY CODE:

```
// Aggregate: Find users with 3+ repeated failed logins
db.event_logs.aggregate([
  { $match: { event_type: "failed-login" } },
  { $group: { _id: "$user_id", failed_count: { $sum: 1 } } },
  { $match: { failed_count: { $gte: 3 } } }
]);
```

EXECUTION OUTPUT:

```
[
  { "_id": "USR202", "failed_count": 3 }
]
```

3. Mini Blogging Platform

Create a blog collection storing post details, authors, tags, and comments. Retrieve posts using tag combinations ($all vs $in) and category filters.

MONGODB QUERY CODE:

```
// Query: Blog in "Technology" category with ALL tags ["NoSQL", "MongoDB"]
db.blogs.find({ category: "Technology", tags: { $all: ["NoSQL", "MongoDB"] } });
```

EXECUTION OUTPUT:

```
{
  "blog_id": "BLOG101",
  "title": "Getting Started with NoSQL & MongoDB",
  "author": { "name": "Ananya Roy", "email": "ananya@example.com" },
  "category": "Technology",
  "tags": [ "NoSQL", "Database", "MongoDB" ],
  "is_published": true
}
```

4. Flexible Product Catalog

Store diverse item categories (laptops, mobiles, cameras) with unique attribute structures inside a polymorphic collection. Perform embedded property queries.

MONGODB QUERY CODE:

```
// Query: Find laptops with RAM = "16GB"
db.products.find({ category: "laptop", "attributes.ram": "16GB" });
```

EXECUTION OUTPUT:

```
{
  "product_id": "P001",
  "name": "Dell XPS 15",
  "category": "laptop",
  "price": 150000,
  "attributes": { "ram": "16GB", "storage": "1TB SSD", "processor": "Intel i7" }
}
```

5. E-Commerce Order Management System

Manage customers and order collections. Use aggregation pipeline ($unwind, $group) to retrieve customer purchase history and calculate order grand totals dynamically.

MONGODB QUERY CODE:

```
// Aggregate: Order calculation for CUST501
db.orders.aggregate([
  { $match: { customer_id: "CUST501" } },
  { $unwind: "$items" },
  {
    $group: {
      _id: "$_id",
      order_id: { $first: "$order_id" },
      calculated_total: { $sum: { $multiply: ["$items.quantity", "$items.unit_price"] } }
    }
  }
]);
```

EXECUTION OUTPUT:

```
[
  { "_id": ObjectId("6724a1b8e4b0123456789a05"), "order_id": "ORD901",
"calculated_total": 5100 }
]
```

6. Website Analytics System

Log web page traffic data (page URL, device type, location, user ID). Perform aggregation to determine top visited pages and most popular device types.

MONGODB QUERY CODE:

```
// Aggregate: Top Visited Pages
db.page_visits.aggregate([
  { $group: { _id: "$page", visit_count: { $sum: 1 } } },
  { $sort: { visit_count: -1 } },
  { $limit: 2 }
]);
```

EXECUTION OUTPUT:

```
[
  { "_id": "/home", "visit_count": 3 },
  { "_id": "/products", "visit_count": 1 }
]
```

7. Content Management System (CMS)

Store publishing records (articles, authors, categories, status). Filter content by draft vs published state using criteria combinations.

MONGODB QUERY CODE:

```
// Query: Retrieve Draft Articles
db.articles.find({ status: { $ne: "published" } });
```

EXECUTION OUTPUT:

```
{
  "article_id": "ART02",
  "title": "Quantum Computing Horizons",
  "author": "Vikram Singh",
  "category": "Research",
  "status": "draft"
}
```

8. Shopping Cart System

Manage cart sub-documents: adding items ($push), updating quantities ($set with positional operators), removing items ($pull), and calculating cart total.

MONGODB QUERY CODE:

```
// Query: Calculate Cart Total Value
db.carts.aggregate([
  { $match: { user_id: "USER_88" } },
  { $unwind: "$cart_items" },
  { $group: { _id: "$user_id", total_cart_value: { $sum: { $multiply:
["$cart_items.price", "$cart_items.quantity"] } } } }
]);
```

EXECUTION OUTPUT:

```
[
  { "_id": "USER_88", "total_cart_value": 5998 }
]
```

9. Product Review and Rating System

Store individual user ratings and feedback. Compute overall metrics (average rating rounded to 2 decimals, total review count) per product.

MONGODB QUERY CODE:

```
// Aggregate: Average Rating and Total Review Count
db.reviews.aggregate([
  { $group: { _id: "$product_id", average_rating: { $avg: "$rating" }, total_reviews: {
$sum: 1 } } },
  { $project: { product_id: "$_id", average_rating: { $round: ["$average_rating", 2] },
total_reviews: 1, _id: 0 } }
]);
```

EXECUTION OUTPUT:

```
[
  { "product_id": "PROD_99", "average_rating": 4.5, "total_reviews": 2 }
]
```

10. Inventory Monitoring System

Compare current stock level with reorder thresholds ($expr with $lte) to automatically isolate products requiring urgent supplier restock.

MONGODB QUERY CODE:

```
// Query: Products requiring immediate restocking
db.inventory.find({ $expr: { $lte: ["$current_stock", "$min_stock_level"] } });
```

EXECUTION OUTPUT:

```
{
  "product_id": "INV01",
  "name": "Monitor 27 inch",
  "current_stock": 5,
  "min_stock_level": 10,
  "supplier": "TechSupplies"
}
```

11. Bank Wallet Transfer System (Transactions)

Perform multi-document ACID financial operations inside MongoDB using client sessions, commit/abort handlers, and conditional field increments ($inc).

MONGODB QUERY CODE:

```
const session = db.getMongo().startSession();
session.startTransaction();
try {
  db.wallets.updateOne({ user_id: "USER_A" }, { $inc: { balance: -1500 } }, { session });
  db.wallets.updateOne({ user_id: "USER_B" }, { $inc: { balance: 1500 } }, { session });
  session.commitTransaction();
  print("Transfer completed successfully.");
} catch (error) { session.abortTransaction(); }
```

CONSOLE OUTPUT:

```
Transfer completed successfully.
Updated USER_A balance: 3500
Updated USER_B balance: 2700
```

12. User Notification System

Store multi-type alerts (order, system, messages). Query unread records for specific users and execute batch status updates (updateMany) upon viewing.

MONGODB QUERY CODE:

```
// Query & Operation: Mark notifications read
db.notifications.updateMany(
  { user_id: "USR55", is_read: false },
  { $set: { is_read: true } }
);
```

EXECUTION OUTPUT:

```
{ "acknowledged": true, "matchedCount": 2, "modifiedCount": 2 }
```

13. Course Learning Platform

Track student module completion percentages. Filter course enrollees by specific progress metrics (100% completion vs range filters $gte/$lte).

MONGODB QUERY CODE:

```
// Query: Retrieve students with 100% progress
db.student_progress.find({ course_id: "CRS_CS101", completion_percentage: 100 });
```

EXECUTION OUTPUT:

```
{
  "student_id": "STU302",
  "course_id": "CRS_CS101",
  "completion_percentage": 100,
  "status": "Completed"
}
```

14. Real-Time Order Tracking System

Model state progression (Placed -> Confirmed -> Shipped -> Delivered). Append historical audit trails ($push) while atomically updating the current status.

MONGODB QUERY CODE:

```
// Query: Retrieve current order status
db.order_tracking.findOne({ order_id: "ORD_7789" }, { order_id: 1, current_status: 1,
_id: 0 });
```

EXECUTION OUTPUT:

```
{ "order_id": "ORD_7789", "current_status": "Shipped" }
```

15. Flexible News Database

Handle schema-less document structures. Use $exists operators to check for dynamic field presence and $or conditions across varying structure parameters.

MONGODB QUERY CODE:

```
// Query: Check field existence ($exists)
db.news.find({ author: { $exists: true } });
```

EXECUTION OUTPUT:

```
{
  "article_id": "NEWS01",
  "title": "Global Climate Summit 2026",
  "author": "Elena Rostova",
  "category": "Environment"
}
```